BaSIC Hyperparameter Benchmark

Complete documentation of the experimental pipeline

Clémence Métayer

April 7, 2026

Contents

1 Introduction **2**

2 Test ODE systems **2**

3 Experimental design **3**
3.1 OFAT approach . . . 3
3.2 Baseline configurations . . . 3
3.3 Varied hyperparameters . . . 3
3.4 Sorting by JAX compilation signature . . . 4

4 Pipeline architecture **5**
4.1 Main orchestrator: `run_all_experiments.py` . . . 5

5 Evaluation metrics **6**
5.1 Structure recovery . . . 6
5.2 Parameter estimation . . . 6
5.3 Credible interval coverage . . . 6
5.4 Predictive quality . . . 7
5.5 MCMC diagnostics . . . 7
5.6 Computational cost . . . 7
5.7 Log-likelihood (post-hoc patch) . . . 7

6 Produced results **7**
6.1 Per-experiment output structure . . . 7
6.2 Global results CSV . . . 8
6.3 `benchmark_metrics.json` file structure . . . 8

7 Visualization and analysis **9**
7.1 System trajectories . . . 9
7.2 Per-hyperparameter heatmaps . . . 9
7.3 Per-parameter ranking . . . 9

8 Hyperparameter inventory **10**

9 Pipeline file summary **11**

10 Reproducibility **11**

1 Introduction

This document describes the complete hyperparameter benchmarking pipeline for the BASIC method (*Bayesian Sparse Identification of Coupled dynamical systems*). The objective is to systematically evaluate the impact of each hyperparameter on the performance of dynamical system identification, in terms of:

- structure recovery (which terms are selected),
- parameter estimation,
- predictive quality,
- MCMC diagnostics (convergence),
- computational cost.

The benchmark follows an OFAT (*One-Factor-At-a-Time*) design around two baseline configurations, corresponding to the two priors implemented in BASIC: the regularized *horseshoe* and the *spike-and-slab*. Each hyperparameter is varied independently from the others, enabling direct sensitivity analysis. Ajouter une référence bibliographique pour la méthode OFAT. In total, the benchmark comprises:

- 105 hyperparameter configurations,
- 5 test ODE systems,

so, in total, **525 experiments** (105×5).

2 Test ODE systems

The benchmark evaluates BASIC on five ordinary differential equation (ODE) systems of increasing complexity, covering different application domains.

Table 1: ODE systems used in the benchmark.

System	**Species**	**Conditions**	**Domain**
Lotka-Volterra	2	1	Ecology (predator-prey)
Chain	2	1	Linear cascade
SEIR	4	2	Epidemiology
Goldbeter	3	1	Oscillatory biology
Yeast Glycolysis	7	1	Metabolism (glycolysis)

Ajouter une référence bibliographique pour chaque système. Il serait bien aussi d'ajouter des détails sur la complexité des exemples.

These systems are defined in YAML files (directory `systems/`) containing:

- the initial conditions `x0` and test initial conditions `test_x0`,
- the true parameters `true_params` (per condition),
- the true SINDy coefficients `true_coeffs_visu` (serving as ground truth),
- the time configuration (t_0, t_{end}, T points),
- the observation parameters (noise, replicates).

The script `visualization/plot_trajectories.py` generates a visualization of the trajectories of each system for all conditions, with trajectories from x_0 (solid lines) and from x_0^{test} (dashed lines).

3 Experimental design

3.1 OFAT approach

Étoffer toute cette section pour que l'on comprenne mieux les choix et leurs implications. N'oublie pas d'ajouter des références bibliographiques pour la méthode OFAT et les alternatives (factorial design, random search, etc.). The experimental plan follows an OFAT (*One-Factor-At-a-Time*) approach: starting from two baseline configurations, each hyperparameter is varied individually while all others remain at their default value. This approach enables:

- direct attribution of the observed effect to the varied parameter,
- a controlled experiment size ($\mathcal{O}(\sum_i n_i)$ instead of $\mathcal{O}(\prod_i n_i)$),
- straightforward interpretation through comparative heatmaps.

The plan is generated by the script `config/generate_plan.py`.

3.2 Baseline configurations

Two baselines are defined, one per prior type:

Table 2: Baseline configurations.

Category	Parameter	Horseshoe (HS)	Spike-and-Slab (SS)
Prior	`type`	horseshoe	spike_and_slab
	`tau0` / `spike_sd`	1.0	0.01
	`slab_scale` / `slab_sd`	1.0	1.0
	`degree_penalty`	1.0	1.0
MCMC	`num_warmup`	2000	2000
	`num_samples`	1000	1000
	`num_chains`	4	4
	`thinning`	1	1
	`target_accept`	0.85	0.85
	`max_treedepth`	6	6
Selection	`strategy`	ci_nonzero_or	incl_prob
	`threshold` / `hyperparameter`	0.0 / 0.95	0.9 / —
Library	`degree`	2	2
	`include_bias`	(system default)	(system default)
	`include_mm`	(system default)	(system default)
Integrator	`shooting`	multiple	multiple
	`segments`	5	5
	`rtol`	10^{-7}	10^{-7}
	`atol`	10^{-9}	10^{-9}
	`max_steps`	4000	4000
Shooting	`shooting_scale`	0.02	0.02
Initialization	`strategy`	regression	regression
	`std_noise`	0.01	0.01
Data	`noise_fraction`	0.1	0.1
	`replicates`	4	4
	`T`	30	30

3.3 Varied hyperparameters

Table 3 summarizes all tested variations. For parameters shared by both priors (MCMC, integrator, library, data, initialization), each value is tested twice: once from the HS baseline, once from the SS baseline.

Table 3: Summary of tested hyperparameter variations.

Category	**Parameter**	**Tested values**	**Count**
Baselines	—	HS + SS	2
HS Prior	`tau0`	0.1, 0.5, 2.0	3
	`slab_scale`	None, 5.0	2
	`degree_penalty`	0	1
SS Prior	`spike_sd`	0.001, 0.1	2
	`slab_sd`	0.5, 5.0	2
	`theta_a, theta_b`	$(1, 5)$, $(0.5, 0.5)$	2
	`degree_penalty`	0	1
MCMC (×2)	`num_warmup`	500, 1000, 5000, 10000	8
	`num_samples`	250, 500, 2000, 5000	8
	`num_chains`	2, 6, 8	6
	`thinning`	2, 5	4
	`target_accept`	0.80, 0.95	4
	`max_treedepth`	8, 10	4
Selection	CI level (HS)	0.90, 0.99	2
	Inclusion threshold (SS)	0.8, 0.95	2
Library (×2)	`degree`	3	2
	`include_bias`	True, False	4
	`include_mm`	True, False	4
Integrator (×2)	`shooting`	single	2
	`segments`	10, 15	4
	`rtol, atol`	$(10^{-5}, 10^{-7})$, $(10^{-9}, 10^{-11})$	4
	`max_steps`	2000, 8000	4
Shooting (×2)	`shooting_scale`	0.01, 0.05, 0.1, 0.2	8
Init. (×2)	`strategy`	median	2
	`std_noise`	0.001, 0.1	4
Data (×2)	`noise_fraction`	0.05, 0.2, 0.3	6
	`replicates`	2, 8	4
	`T`	15, 50	4
Total			**105**

3.4 Sorting by JAX compilation signature

To optimize execution time, configurations are sorted to maximize JAX compilation cache reuse. Parameters that affect compilation (array shapes, static arguments) are grouped into a *compilation signature*:

- `library.degree`, `library.include_bias`, `library.include_mm`
- `integrator.shooting`, `integrator.segments`, `integrator.max_steps`
- `mcmc.num_chains`
- `observations.replicates`, `time.T`

Experiments with the same signature are executed consecutively, thereby avoiding costly recompilations.

4 Pipeline architecture

The benchmark pipeline is divided into four main stages:

1. **Plan generation** (`config/generate_plan.py`)
   → `config/experiment_plan.csv` (105 configurations)
2. **Execution** (`run_all_experiments.py`)
   For each (system, configuration):
   a. Build a temporary YAML (system base + overrides)
   b. Run BASIC (sparse inference + refit)
   c. Compute metrics (`metrics/benchmark_metrics.py`)
   → `all_benchmark_results.csv` (525 rows)
3. **Post-hoc patch** (`metrics/patch_log_likelihood.py`)
   Add log-likelihood to existing results
4. **Visualization and analysis**

   a. `visualization/plot_trajectories.py` → system trajectories
   b. `visualization/analyze_recommendations.py` → heatmaps + rankings

4.1 Main orchestrator: `run_all_experiments.py`

This script manages the complete benchmark execution. Its main features are:

YAML configuration construction. For each experiment, a YAML file is built by merging the system base configuration (e.g. `systems/lotka_volterra.yaml`) with the hyperparameter overrides from the CSV plan. A mapping dictionary translates the flat CSV columns into nested YAML keys:

```
"sparse_prior.tau0"  -> ("sparse_prior", "tau0")
"mcmc.num_warmup"    -> ("mcmc", "num_warmup")
```

Keys specific to the unused prior are automatically removed (e.g. `tau0` and `slab_scale` are removed if the prior is *spike-and-slab*).

Resumable execution. The existence of the `benchmark_metrics.json` file in the output directory serves as a completion marker. Already completed experiments are automatically skipped, enabling resumption after interruption.

Parallelism. The script supports parallel execution via `ProcessPoolExecutor` (argument `-workers`). A shared JAX compilation cache (`.jax_cache/`) is used by all subprocesses.

Global CSV reconstruction. After execution, the global CSV `all_benchmark_results.csv` is entirely rebuilt from individual `benchmark_metrics.json` files, avoiding duplicates.

Command-line options.

```
python run_all_experiments.py
    --systems lotka_volterra chain   # filter systems
    --runs 0 1 2                     # filter run_ids
    --workers 4                      # parallel execution
    --skip-refit                     # sparse inference only
    --skip-metrics                   # skip metrics computation
    --plan custom_plan.csv           # custom plan
    --python /path/to/python         # Python interpreter
```

5 Evaluation metrics

The script `metrics/benchmark_metrics.py` computes a comprehensive set of metrics for each experiment. These metrics are organized into five categories.

5.1 Structure recovery

The true structure matrix $M_{\text{true}} \in \{0,1\}^{n_{\text{eq}} \times n_{\text{feat}}}$ is built from `true_coeffs_visu` in the YAML, where $M_{\text{true}}[j,k] = 1$ if term k appears in equation j of the true system. The learned matrix $\hat{M}$ is built from `structure.json` (the result of BASIC's variable selection).

F1-score. The matrices are flattened and compared element-wise:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{1}$$

where TP (true positives), FP (false positives), TN (true negatives) and FN (false negatives) are counted over all matrix elements.

Normalized Hamming distance.

$$d_H = \frac{1}{n_{\text{eq}} \cdot n_{\text{feat}}} \sum_{j,k} \mathbf{1}[M_{\text{true}}[j,k] \neq \hat{M}[j,k]] \tag{2}$$

$d_H = 0$ indicates a perfectly identified structure, $d_H = 1$ a completely different structure.

5.2 Parameter estimation

NMSPE (Normalized Mean Squared Parameter Error). The true SINDy coefficients Ξ_{true} are extracted from the YAML. The learned coefficients $\hat{\Xi}$ are the median of the MCMC samples. The NMSPE is computed on active terms only (where $M_{\text{true}} = 1$):

$$\text{NMSPE}_{\text{active}} = \frac{\frac{1}{|\mathcal{A}|} \sum_{(j,k)\in\mathcal{A}} (\hat{\Xi}_{jk} - \Xi_{\text{true},jk})^2}{\frac{1}{|\mathcal{A}|} \sum_{(j,k)\in\mathcal{A}} \Xi^2_{\text{true},jk}} \tag{3}$$

where $\mathcal{A} = \{(j,k) : M_{\text{true}}[j,k] = 1\}$.

FP_MSE. Mean squared error on false positive terms (where $M_{\text{true}} = 0$):

$$\text{FP_MSE} = \frac{1}{|\bar{\mathcal{A}}|} \sum_{(j,k)\notin\mathcal{A}} \hat{\Xi}^2_{jk} \tag{4}$$

This measures the magnitude of learned coefficients where the true coefficient is zero.

5.3 Credible interval coverage

For a level α (default 95%), the credible interval bounds are computed from the quantiles of the MCMC samples:

$$CI_{jk} = \left[Q_{\alpha/2}(\Xi^{(1:S)}_{jk}),\ Q_{1-\alpha/2}(\Xi^{(1:S)}_{jk}) \right] \tag{5}$$

The coverage on active terms is the fraction of true coefficients falling within their CI:

$$\text{Coverage}_{\text{active}} = \frac{1}{|\mathcal{A}|} \sum_{(j,k)\in\mathcal{A}} \mathbf{1}\left[\Xi_{\text{true},jk} \in CI_{jk}\right] \tag{6}$$

A well-calibrated Bayesian model should have coverage close to α.

The **mean width** of the CIs on active terms is also reported as a measure of uncertainty.

5.4 Predictive quality

NMSE on training data. The learned model (with median coefficients) is simulated from the training initial conditions, and the NMSE is computed between predicted trajectories and observed data (averaged over replicates if necessary):

$$\mathrm{NMSE}_{\mathrm{train}} = \frac{\sum_{t,i}(y_{\mathrm{pred},t,i} - \bar{y}_{\mathrm{obs},t,i})^2}{\sum_{t,i} \bar{y}^2_{\mathrm{obs},t,i}} \tag{7}$$

NMSE on a new initial condition. To test generalization ability, the learned model is simulated from a new initial condition x_0^{new} (defined in the YAML as `test_x0`, or otherwise a $+10\%$ perturbation of x_0). The trajectory is compared to that of the true system:

$$\mathrm{NMSE}_{\mathrm{new}\ x_0} = \frac{\|x_{\mathrm{pred}} - x_{\mathrm{true}}\|_F^2}{\|x_{\mathrm{true}}\|_F^2} \tag{8}$$

The simulation uses the `Tsit5` integrator from Diffrax with a maximum of 10 000 steps.

5.5 MCMC diagnostics

Convergence diagnostics are extracted from the `convergence_diagnostics.json` file produced by BASIC:

- $\hat{R}_{\max}$ (`rhat_max_sparse`): maximum $\hat{R}$ value across all sites. Should be < 1.01 for reliable convergence.
- $\mathrm{ESS}_{\min}$ (`ess_min_sparse`): minimum bulk effective sample size. Low values indicate inefficient exploration.

5.6 Computational cost

Three timings are recorded (in seconds):

- `time_sparse_s`: duration of sparse inference (step 1),
- `time_refit_s`: duration of the refit (step 2, summed across conditions),
- `time_total_s`: total duration including visualization.

5.7 Log-likelihood (post-hoc patch)

The Gaussian log-likelihood is computed post-hoc by the script `metrics/patch_log_likelihood.py`, without re-running MCMC. For each sample s and each condition c:

$$\log p(y \mid \theta^{(s)}) = \sum_{t,i} \left[-\frac{1}{2}\log(2\pi) - \log \sigma_i^{(s,c)} - \frac{1}{2}\left(\frac{y_{t,i} - \hat{y}_{t,i}^{(s,c)}}{\sigma_i^{(s,c)}} \right)^2 \right] \tag{9}$$

where $\hat{y}^{(s,c)}$ comes from the posterior predictive draws (`X_post_draws`) and $\sigma^{(s,c)}$ from the inferred noise standard deviation. The reported statistics are the median, mean, and standard deviation over MCMC samples.

6 Produced results

6.1 Per-experiment output structure

Each experiment produces a directory `out_<system>_run<id>/` containing:

- `run_config.json`: hyperparameters used for this experiment,
- `config.yaml`: complete generated YAML configuration,

- `benchmark_metrics.json`: all metrics (hierarchical JSON structure),
- `timings_pipeline.json`: computational timings,
- `experiment.log`: BASIC standard output/error,
- `multi_cond_sparse_inference/`: sparse inference artifacts,
  - `artifacts_sparse.npz`: MCMC samples, posterior predictions,
  - `structure.json`: variable selection result,
  - `convergence_diagnostics.json`: $\hat{R}$, ESS per site,
  - `coefficients_mean_C*.csv`: median coefficients per condition,
  - `selected_terms.csv`: binary table of selected terms,
  - `meta.json`: metadata (species, features, etc.),
- `refit/`: refit artifacts (one set per condition),
  - `artifacts_refit_C*.npz`, `convergence_diagnostics_refit_C*.json`, etc.

6.2 Global results CSV

The file `all_benchmark_results.csv` (525 rows) aggregates all metrics into a flat table with the following columns:

Table 4: Columns of the global results CSV.

Category	**Columns**
Identification	`run_id`, `system_name`, `category`, `description`
Structure	`F1`, `Precision`, `Recall`, `TP`, `FP`, `TN`, `FN`, `Hamming`
Parameters	`NMSPE_active`, `NMSPE_all`, `FP_MSE`, `n_active_terms`
CI coverage	`CI_coverage_active`, `CI_coverage_all`, `CI_mean_width_active`
Prediction	`NMSE_training`, `NMSE_new_x0`
Diagnostics	`rhat_max_sparse`, `ess_min_sparse`
Timing	`time_sparse_s`, `time_refit_s`, `time_total_s`
Metadata	`n_mcmc_samples`, `n_species`, `n_features`

6.3 `benchmark_metrics.json` file structure

This JSON file adopts a hierarchical structure:

```
{
  "metadata": {timestamp, system_name, outdir, n_mcmc_samples, ...},
  "structure": {
    "f1": {TP, FP, TN, FN, Precision, Recall, F1},
    "hamming": float,
    "M_true_active": int, "M_hat_selected": int
  },
  "parameters": {NMSPE_active, NMSPE_all, FP_MSE, ...},
  "ci_coverage": {CI_coverage_active, CI_coverage_all,
     CI_mean_width_active, ...},
  "training_fit": {NMSE_training, MSE_training, n_points},
  "prediction": {NMSE, MSE, normalization, x0_new, success},
  "diagnostics": {rhat_max_sparse, ess_min_sparse},
  "timings": {step1_sparse_s, step2_refit_s, total_with_viz_s},
  "log_likelihood": {log_lik_median, log_lik_mean, log_lik_std}
}
```

7 Visualization and analysis

7.1 System trajectories

The script `visualization/plot_trajectories.py` generates a matrix figure ($5 \times n_{\text{cond}}$) showing the trajectories of each system for each experimental condition. Trajectories from x_0 are plotted as solid lines, those from x_0^{test} as dashed lines. The figure is saved to `figures/system_trajectories.png`.

7.2 Per-hyperparameter heatmaps

The script `visualization/analyze_recommendations.py` produces 24 heatmaps (one per parameter group, listed in Table 5) saved to `figures/heatmaps/`.

Each heatmap is organized as follows:

- **Rows**: the baseline (in bold) followed by variations, first for the HS prior then for the SS prior (separated by a black line).
- **Columns**: the 5 ODE systems.
- **Subplots**: arranged in a 3-column grid, one per metric (F1, Precision, Recall, Hamming, NMSPE, FP_MSE, NMSE training, NMSE new x_0, log-likelihood, $\hat{R}_{\max}$, ESS min, total time).
- **Color**: sequential palette per metric category (Blues for structure, Oranges for estimation, Greens for prediction, etc.). Light = low value, dark = high value.
- **Text**: raw numerical value with adaptive precision.

Table 5: Parameter groups analyzed by heatmap.

Filename	**Title**	**Priors**
`heatmap_segments`	Segments (integrator)	HS + SS
`heatmap_max_steps`	Max steps (integrator)	HS + SS
`heatmap_chains`	MCMC chains	HS + SS
`heatmap_replicates`	Replicates (data)	HS + SS
`heatmap_T`	Time horizon T	HS + SS
`heatmap_tau0`	τ_0	HS only
`heatmap_slab_scale`	Slab scale	HS only
`heatmap_spike_sd`	Spike SD	SS only
`heatmap_slab_sd`	Slab SD	SS only
`heatmap_theta`	θ_a, θ_b	SS only
`heatmap_degree_penalty`	Degree penalty	HS + SS
`heatmap_warmup_samples`	Warmup & Samples	HS + SS
`heatmap_thinning`	Thinning	HS + SS
`heatmap_target_accept`	Target accept	HS + SS
`heatmap_max_treedepth`	Max treedepth	HS + SS
`heatmap_ci_level`	CI level	HS only
`heatmap_incl_prob`	Inclusion threshold	SS only
`heatmap_rtol_atol`	ODE tolerances	HS + SS
`heatmap_shooting_scale`	Shooting scale	HS + SS
`heatmap_init_median`	Init strategy (median)	HS + SS
`heatmap_init_std_noise`	Initialization noise	HS + SS
`heatmap_noise`	Noise fraction	HS + SS
`heatmap_single_shooting`	Single shooting	HS + SS
`heatmap_library`	SINDy library	HS + SS

7.3 Per-parameter ranking

The script also produces an automatic ranking of the best configurations for each hyperparameter, saved to `figures/heatmaps/best_config_per_parameter.csv`.

Ranking method. For each hyperparameter and each prior type:

1. The tested configurations (including the baseline) are compared.
2. For each (metric, system) pair, configurations are ranked: rank 1 = best value. The direction (ascending/descending) depends on the metric (e.g. F1: descending, NMSPE: ascending).
3. The mean rank is computed across all (metric, system) pairs.
4. The configuration with the lowest mean rank is declared best.

The metrics used for ranking exclude computational cost, thus covering structure, estimation, prediction, log-likelihood, and MCMC diagnostics.

8 Hyperparameter inventory

The file `config/hyperparameter_methods.csv` documents the 32 hyperparameters of BASIC, of which 31 are included in the benchmark. Table 6 provides a summary.

Table 6: Complete inventory of BASIC hyperparameters.

Category	Name	Role	Default
Library	`library.degree`	Max degree of polynomial monomials	2
	`library.include_bias`	Constant term in the library	False
	`library.include_mm`	Michaelis-Menten terms $\frac{x_i}{K+x_j}$	False
Prior selection	`sparse_prior.type`	horseshoe or spike_and_slab	horseshoe
HS Prior	`sparse_prior.tau0`	Global HalfCauchy scale on τ	1.0
	`sparse_prior.slab_scale`	Regularized slab scale (Piironen & Vehtari)	None
	`sparse_prior.degree_penalty`	Exponent p: $\lambda_k \sim$ HalfCauchy$(1/d^p)$	1.0
SS Prior	`sparse_prior.spike_sd`	Spike SD (near-zero coefficients)	0.01
	`sparse_prior.slab_sd`	Slab SD (non-zero coefficients)	1.0
	`sparse_prior.theta_a`	α of the Beta prior on θ_k	1.0
	`sparse_prior.theta_b`	β of the Beta prior on θ_k	1.0
MCMC/NUTS	`mcmc.num_warmup`	Adaptation iterations	(YAML)
	`mcmc.num_samples`	Posterior samples per chain	(YAML)
	`mcmc.num_chains`	Number of parallel chains	4
	`mcmc.thinning`	Thinning (keep 1 out of N)	1
	`mcmc.target_accept`	NUTS target acceptance probability	0.8
	`mcmc.max_treedepth`	Max tree depth (2^d leapfrog steps)	6
ODE solver	`integrator.shooting`	Method: single or multiple	single
	`integrator.segments`	Number of segments (multiple shooting)	None
	`integrator.rtol`	Relative tolerance (PIDController)	10^{-7}
	`integrator.atol`	Absolute tolerance	10^{-9}
	`integrator.max_steps`	Max solver steps	adaptive
Shooting	`shooting_scale`	Continuity constraint SD	0.02
Selection	`selection.strategy`	Term selection strategy	ci_nonzero_global
	`selection.threshold`	Threshold (method-dependent)	0.0

Continued on next page

Category	Name	Role	Default
	`selection.hyperparameter`	Hyperparameter (e.g. CI level)	0.95
Initialization	`init.strategy`	regression or median	regression
	`init.std_noise`	Noise added to initialization	0.01
Data	`observations.noise_fraction`	Fraction of per-species max for noise	(YAML)
	`observations.replicates`	Number of biological replicates	(YAML)
	`time.T`	Number of time points	(YAML)

9 Pipeline file summary

Table 7: Main pipeline files.

File	Role
`config/generate_plan.py`	Generates the OFAT experiment plan (105 configs)
`config/experiment_plan.csv`	Experiment plan (benchmark input)
`config/hyperparameter_methods.csv`	Documented inventory of 32 hyperparameters
`run_all_experiments.py`	Orchestrator: runs all experiments
`metrics/benchmark_metrics.py`	Computes evaluation metrics
`metrics/patch_log_likelihood.py`	Adds log-likelihood post-hoc
`visualization/plot_trajectories.py`	Trajectories of the 5 ODE systems
`visualization/analyze_recommendations.py`	Heatmaps + per-parameter ranking
`all_benchmark_results.csv`	Aggregated results (525 rows)
`figures/system_trajectories.png`	Trajectory figure
`figures/heatmaps/*.png`	24 heatmaps per parameter group
`figures/heatmaps/best_config_per_parameter.csv`	Best configuration ranking

10 Reproducibility

To reproduce the complete benchmark:

1. **Generate the experiment plan**:

```
python config/generate_plan.py
```

2. **Run the experiments** (sequential or parallel execution):

```
python run_all_experiments.py --workers 4
```

3. **Patch log-likelihood**:

```
python metrics/patch_log_likelihood.py
```

4. **Generate visualizations**:

```
python visualization/plot_trajectories.py
python visualization/analyze_recommendations.py
```

Execution is automatically resumable: already completed experiments (presence of `benchmark_metrics.jso` are skipped. The global CSV is rebuilt at each launch.